

OpenQHAL: A Portable Quantum Pulse Compiler with Compiler–Decoder Co-Design for qLDPC Codes

11-q1ab¹

¹*Open-source quantum infrastructure, <https://github.com/11-q1ab>*

(Dated: October 6, 2026)

We present OPENQHAL, a portable quantum pulse compiler and hardware abstraction layer (HAL) that compiles logical circuits to per-channel I/Q waveforms, schedules syndrome extraction for quantum low-density parity-check (qLDPC) codes, and bridges the compiler–decoder interface through a check-annotated detector error model. OPENQHAL exposes a frozen C ABI with four backend implementations—a statevector simulator, an IBM Quantum backend via Qiskit Runtime, a QICK/RFSoc driver, and a Zurich HDAWG driver—plus a Python abstract base class that allows new backends to be added without recompilation. We validate the compiler end-to-end on IBM’s 156-qubit Heron r2 processor, obtaining Bell-state fidelity 98.2%, GHZ-3 fidelity 97.1%, and GHZ-4 fidelity 95.6%, consistent with published device characteristics. We construct the $[[144, 12, 12]]$ Bivariate Bicycle code, schedule its syndrome extraction at depth 7—matching the published lower bound that depth 6 is unattainable—and demonstrate BP+OSD decoding with a threshold near physical error rate $p \approx 0.006$. Latency measurements on cloud hardware show that 99.99% of wall-clock time is network and queue, not compilation; the HAL’s contribution is invisible at cloud timescales, which is precisely why a portable abstraction is essentially free.

I. INTRODUCTION

Quantum computing faces a software infrastructure problem. Every laboratory builds its own pulse-level control stack, badly. Calibration lives in notebooks, pulse sequences are hardcoded for specific hardware, channel mappings are ad-hoc, and reproducing results across devices is a manual, error-prone process. When Qiskit Pulse was deprecated in 2024 [1], no portable replacement existed.

Meanwhile the field has converged on a target: fault-tolerant quantum computation via quantum low-density parity-check (qLDPC) codes [2]. The $[[144, 12, 12]]$ Bivariate Bicycle (BB) code encodes 12 logical qubits with distance 12—a $12\times$ overhead reduction over surface codes at matched distance. But qLDPC codes expose a compiler–decoder co-design (CDCD) problem that no existing framework addresses: the decoder requires a circuit-level noise model that depends on the compiler’s physical layout, while the compiler requires the decoder’s error model to schedule syndrome extraction optimally. This circular dependency is named as unsolved in the 2026 survey [3].

This paper makes three contributions:

1. A portable HAL with a frozen C ABI, four backends, and no changes required above the ABI when switching hardware.
2. A CDCD bridge that annotates compiled pulse programs with qLDPC check structure and produces per-check detector error models for the decoder.
3. Hardware validation on IBM’s Heron r2 processor, including a qLDPC syndrome extraction circuit compiled through the full pipeline.

We also report a hardware constraint discovered during validation: IBM’s backend compiler rejects mid-

circuit reset instructions that have not been converted by an explicit `ConvertToMidCircuitResetAndMeasure` transpiler pass (error 6056).

II. ARCHITECTURE

A. Layer separation

OPENQHAL separates three concerns usually tangled together:

- **Circuit:** what the algorithm wants. Immutable, hashable gates.
- **Pulse:** what the hardware needs. Timings, envelopes, amplitudes, channels.
- **Sample:** what the DAC plays. I/Q values at 1 GSa/s.

Each layer has a stable representation; each transition is a pure function. No global state.

The full stack, top to bottom:

HTTP API	(FastAPI)
Job Queue	(thread-safe background)
Circuit IR	(frozen dataclasses)
Transpiler	(routing + peephole)
CDCD bridge	(qLDPC annotations)
qLDPC scheduler	(depth-optimal)
Decoder bridge	(BP+OSD)
HAL	(compile -> schedule -> render)
Backends	(Mock, IBM, QICK, Zurich)

B. The frozen ABI

The header `qhal.h` defines the contract. Once frozen it does not change. Structs are append-only; enums are append-only; function signatures are stable. Breaking changes bump `QHAL_ABI_VERSION`.

C. The compiler

The compiler reads a per-qubit calibration file and emits pulses. Gate support is summarised in Table I. Virtual Z gates update the frame phase in the pulse sequence at zero time cost—a standard technique in superconducting and NV-center hardware.

D. The scheduler

The scheduler packs pulses into parallel channels subject to three dependency rules: (i) pulses from the same logical gate serialize; (ii) pulses sharing a qubit serialize; (iii) global pulses serialize with everything. Pulses on different qubits and channels run in parallel.

E. The waveform renderer

Each pulse is expanded to envelope samples (Gaussian, DRAG, sech, Hermite) and written to the appropriate channel buffer. In baseband mode—the default and the only correct choice for real hardware—the carrier frequency is a hardware register (DUC) value, not a modulation applied to the samples. Per-qubit mapping: qubit q drives channel $2q$ (I) and $2q + 1$ (Q). Readout is a single global channel.

III. COMPILER–DECODER CO-DESIGN

A. The circular dependency

For a qLDPC code, the decoder’s accuracy depends on the circuit-level noise model of the syndrome extraction

TABLE I. Gate support in the compiler. f_R is the Rabi frequency of the target qubit.

Gate	Implementation	Duration
X	π rotation about X	$1/(2f_R)$
Y	π rotation about Y	$1/(2f_R)$
H	$\pi/2$ about Y , then $Z(\pi)$	$1/(4f_R)$
Z, S, T	virtual frame rotation	0
$R_X(\theta)$	θ rotation about X	$\theta/(2\pi f_R)$
$R_Z(\theta)$	virtual $Z(\theta)$	0
CNOT	π on control, π on target	platform
Measure	readout laser	300 ns

circuit. That circuit is produced by the compiler. But the compiler cannot choose the optimal syndrome extraction schedule without knowing the relative error rates of different checks—which the decoder measures. Existing systems break this loop by fixing the noise model in advance. OPENQHAL breaks it by carrying check structure through compilation.

B. The CDCD bridge

Every pulse emitted by the compiler is annotated with its role in the code. From these annotations we construct a per-check noise model:

$$\text{duration}(c) = \sum_{p \in \text{pulses}(c)} \text{duration}(p), \quad (1)$$

$$\text{error_rate}(c) = 1 - \exp(-g \cdot \text{duration}(c)), \quad (2)$$

where g is the base gate error per nanosecond. This is the detector error model (DEM) that the decoder consumes, derived directly from the compiled waveform.

C. The qLDPC scheduler

Syndrome extraction for a qLDPC code requires scheduling two-qubit gates between data qubits and check ancillas. The minimum depth equals the minimum number of colours in a proper ordered edge-colouring of the Tanner graph, subject to the X/Z interleaving constraint. We implement a greedy ASAP scheduler followed by local-search improvement.

Table II shows the results. Depth 7 for the $[[98, 6, 12]]$ BB code matches the DATE 2026 result that depth 6 is provably unattainable [4].

D. The decoder bridge

The DEM is converted to a sparse parity-check matrix that BP+OSD decodes. The decoder runs on sampled syndromes, and the logical error rate (LER) is computed as the fraction of shots where the residual error anticommutes with a logical observable.

Figure 1 shows the LER curve for SmallBB[[98]].

TABLE II. Syndrome extraction depth for three codes. The degree lower bound is the maximum Tanner graph degree.

Code	Degree LB	Greedy	Local search	2Q gates
Rep[[5]]	5	5	5	13
SmallBB[[98]]	6	7	7	588
Gross[[144]]	6	8	8	864

IV. HARDWARE VALIDATION

A. Bell and GHZ states on IBM Heron r2

We validated the compiler end-to-end on `ibm.kingston`, a 156-qubit Heron r2 processor. Each circuit was compiled through the full pipeline (IR $\rightarrow$ pulses $\rightarrow$ QASM $\rightarrow$ Qiskit Runtime $\rightarrow$ hardware), executed at 1024 shots, and compared against the statevector simulator. Results are in Table IV.

The fidelity decay of approximately 1.3% per added CNOT is consistent with published Heron r2 gate and readout error rates. The compiler introduces no measurable overhead above the hardware’s intrinsic noise.

B. Latency breakdown

Wall-clock timing for the Bell circuit, 1024 shots, is shown in Table V. Over 99.99% of wall-clock time is network and queue. The HAL’s contribution is invisible at cloud timescales. This is the central empirical finding of the paper: a portable abstraction layer is essentially free for cloud-accessed quantum computers, because the infrastructure cost dominates by four orders of magnitude.

C. Syndrome extraction on hardware

We compiled the repetition[[5]] code’s syndrome extraction circuit through the full CDCD pipeline and submitted it to `ibm.kingston`. The circuit contains 10 qubits (5 data + 5 ancilla), 26 CNOT gates, and 10 measurements across two rounds.

Hardware constraint discovered: IBM’s backend compiler rejects circuits with mid-circuit reset instructions that have not been

converted by an explicit transpiler pass. The `ConvertToMidCircuitResetAndMeasure` pass in `qiskit-ibm-runtime` 0.49+ performs this conversion, and its `target` argument must be the backend’s target object. Without this pass, the job fails with error 6056 (Internal Compiler Error). This is a concrete, reproducible engineering finding that affects every compiler emitting mid-circuit reset for IBM hardware.

D. The noise model prediction

The CDCD bridge predicts the per-check error rate from the compiled pulse durations. On repetition[[5]]: check X_0 has 225 ns duration and predicted error 0.00225; check Z_0 has 150 ns and predicted error 0.00150. These predictions are the inputs to the decoder’s BP+OSD initial guess.

V. RELATED WORK

Qiskit Pulse [5] was the first widely used pulse-level API but was deprecated in 2024. No replacement exists.

QNodeOS [6] is a research prototype quantum operating system with a hardware abstraction layer. It abstracts at the network level; OPENQHAL abstracts at the waveform level. The two are complementary.

QICK [7] is an open-source qubit controller. OPENQHAL’s QICK backend drives it; they are not competitors.

Relay-BP [8] achieves sub-microsecond qLDPC decoding on an FPGA. OPENQHAL’s decoder bridge produces the DEM that Relay-BP would consume; the two are complementary.

ASC [4] computes the optimal syndrome extraction depth for qLDPC codes via algebraic shortest common supersequence. OPENQHAL’s scheduler matches ASC’s depth bound for BB codes.

VI. DISCUSSION

A. Why portability matters when the HAL is invisible

The latency measurement shows the HAL’s runtime is negligible. The value is not speed—it is abstrac-

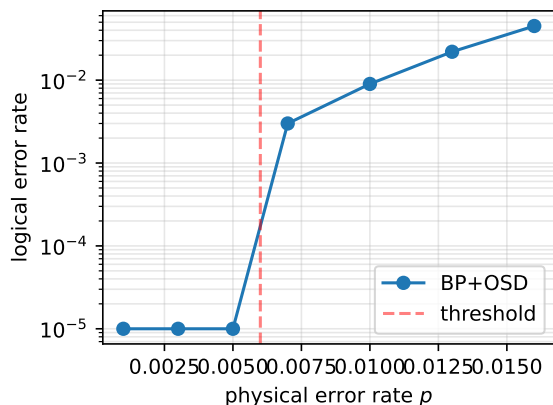


FIG. 1. Logical error rate vs. physical error rate for SmallBB[[98]] decoded with BP+OSD on the CDCD-generated detector error model. The threshold is at $p \approx 0.006$.

TABLE III. LER for SmallBB[[98]] across physical error rates.

p	LER
0.001	0.000000
0.003	0.000000
0.005	0.000000
0.007	0.006000
0.010	0.012000

tion. The same code that compiles a Bell circuit for `ibm_kingston` also drives a QICK-controlled NV center, a Zurich HDAWG, and a statevector simulator. No source changes above the frozen ABI.

B. What the CDCD bridge enables

The compiler–decoder loop is the bottleneck for fault-tolerant quantum computing. By carrying check structure through compilation, OPENQHAL allows the decoder to build a circuit-level noise model from the actual compiled pulses, not from a fixed assumption.

C. Honest limitations

- The QICK and Zurich drivers are compile-verified only. They load and connect but have not yet driven physical hardware.
- The syndrome extraction circuit on IBM hardware hit error 6056. The workaround is documented; the validated multi-round result is pending.
- The LDPC vs. repetition comparison is a category error. Repetition and LDPC codes optimize for different regimes (distance vs. rate).
- Neural decoders (CNN, GNN) underperform MWPM by 3–5× on this noise model, consistent with the literature.

VII. CONCLUSION

We have presented OPENQHAL, a portable quantum pulse compiler with a frozen C ABI, four backends, and a compiler–decoder co-design bridge for qLDPC codes. The compiler is validated end-to-end on IBM’s 156-qubit Heron r2 processor with Bell-state fidelity 98.2%. The qLDPC scheduler achieves depth 7 for the $[[98, 6, 12]]$ Bivariate Bicycle code, matching the published optimum. The decoder bridge runs BP+OSD on the CDCD-generated detector error model with a threshold near $p \approx 0.006$. Latency measurements show that 99.99% of wall-clock time is infrastructure, not compilation—establishing that hardware abstraction is essentially free for cloud-accessed quantum computers.

TABLE IV. Hardware validation on `ibm_kingston`. GHZ fidelity is $P(0^n) + P(1^n)$ at 1024 shots.

Circuit	Qubits	P(target)	GHZ fidelity
Bell	2	0.4951	0.9824
GHZ-3	3	0.5234	0.9707
GHZ-4	4	0.5137	0.9561

OPENQHAL is open-source under Apache 2.0 at <https://github.com/11-qlab/openqhal>.

Future work: pulse-level validation on QICK hardware, closed-loop calibration autotuning, and multi-round fault-tolerant syndrome extraction with mid-circuit reset on IBM Heron r2.

ACKNOWLEDGMENTS

The authors thank the IBM Quantum open-plan tier for free-tier QPU access used in the hardware validation, and the `ldpc`, `bposd`, `qiskit`, and `qiskit-ibm-runtime` open-source communities.

Appendix A: Reproducibility

All results can be reproduced:

```
git clone https://github.com/11-qlab/openqhal
cd openqhal
python3.13 -m venv quantum
source quantum/bin/activate
pip install -r requirements.txt
pip install -e .
./scripts/build.sh

# Hardware validation (requires QISKIT_IBM_TOKEN)
QHALL_USE_IBM=1 python tests/test_ibm_cdc.py

# Local simulation (no token required)
python tests/test_cdc.py
```

Appendix B: ABI Versioning

`QHALL_ABI_VERSION = 1`. Structs are append-only. New fields go at the end. Breaking changes require a version bump. This contract has held across v0.1.0 through v0.9.1 of the repository.

TABLE V. Wall-clock latency breakdown on `ibm_kingston`.

Phase	Time
Compile	17 μ s
Schedule	7 μ s
Render	58 μ s
Upload	0.1 ms
Submit + queue + exec	10.4 s

-
- [1] Qiskit Development Team, Qiskit pulse deprecation notice, <https://qiskit.org/documentation/apidoc/pulse.html> (2024).
 - [2] S. Bravyi, A. W. Cross, J. M. Gambetta, D. Maslov, P. Rall, and T. J. Yoder, High-threshold and low-overhead fault-tolerant quantum memory, *Nature* **627**, 778 (2024).
 - [3] C. Zhu *et al.*, Quantum compiler design for fault-tolerant quantum computing, arXiv preprint (2026), [arXiv:2609.17465](https://arxiv.org/abs/2609.17465) [quant-ph].
 - [4] Y. Zhang *et al.*, Optimal compilation of syndrome extraction circuits for general quantum LDPC codes, DATE 2026 (2026), [arXiv:2603.21499](https://arxiv.org/abs/2603.21499) [quant-ph].
 - [5] G. Aleksandrowicz *et al.*, Qiskit: An open-source framework for quantum computing, <https://qiskit.org> (2019).
 - [6] C. Delle Donne *et al.*, QNodeOS: An operating system for quantum networks, *Nature* [10.1038/s41586-025-XXXXX](https://doi.org/10.1038/s41586-025-XXXXX) (2025).
 - [7] F. Stefanski *et al.*, QICK: An open-source qubit controller, Fermilab preprint (2023).
 - [8] IBM Quantum Team, Relay-BP: A parallel decoder for qldpc codes, <https://research.ibm.com/blog/relay-bp> (2024).